University of Canberra
Faculty of Science and Technology

**Software Technology 1**
**(4483/8995)**

## Week 5 Tutorial & Computer Lab

## Assignment 2 Case Studies with Programming Problems

## Case Study 2: Numpy and OpenCV packages

- This lab activity is assessed and contributes to the Assignment 2 mark.
- Please go through this Week_5_Tutorial_and Computer_Laboratory.doc available in Module 5 in Canvas
- For assessment of this activity, you need to complete the following in week 5:

  **1)** attend your allocated computer lab class in Week 5,

  **2)** complete all tasks listed in this handout,

  **3)** demonstrate the working activities to your tutor in class, and

  **4)** Save your python project to a zip file and upload this zip file to **In-class Lab Work Demo Week 5** and upload it to your GitHub Repository. Make sure you include a commit message with date and time, when you upload this zip file in GitHub.

- Your mark is 0 if you do not demonstrate your project.
- You can leave the class after you complete the 4 requirements above.

### Tasks

- Introduction to Numpy Library/Package (https://numpy.org/ )
- Introduction to OpenCV Python Library/Package ( opencv-python · PyPI)

Download the Week_5_Lab_Activities.zip in Canvas, and unzip the folder. This folder content contains all the necessary activities needed for each task completion in this week.

### Task 1:Introduction to Numpy package (NumPy)

- Go through the documentation fro Numpy (NumPy) to understand what this package can do.
- Numpy need to be installed in your python programming environment as it not include as part of the standard Python library. You need to install it using pip for windows machine using the terminal:

```
>> pip install numpy
```

- **Note:**
  After installing numpy, for each of the activity subtask questions, you need to write a python script and test it, or, test the python script (if the script is provided already) for each of the following questions in Task 1, and save the correctly working scripts as : task1_1.py, task1_2.py……..task1_10.py. The scripts (if not included) , the test results and/or screen shots need to be embedded in this document only and submitted to GitHub, after demonstration in the tutorial class to the tutor

**Task 1_1:** Create a 1D numpy array with the integers from 10 to 19 inclusive, and print the array, its shape, dimension, and data type.

```
#your answer
#task1_1.py

import numpy as np
arr = np.arange(10, 20)
print("Array:", arr)
print("Shape:", arr.shape)
print("Dimension:", arr.ndim)
print("Data type:", arr.dtype)
```

- The expected console output for a successful test run for task1_1.py is as shown below:

```
Array: [10 11 12 13 14 15 16 17 18 19]
Shape: (10,)
Dimension: 1
Data type: int64
```

**Task 1_2:** Create a 3x3 numpy array filled with zeros and then replace the element in the center (row=1, column=1) with 7. Print the resulting array.

```
#your answer
#task1_2.py

import numpy as np

arr = np.zeros((3, 3), dtype=int)
arr[1, 1] = 7
print(arr)
```

- The expected console output for a successful test run for task1_2.py is as shown below:

```
[[0 0 0]
 [0 7 0]
 [0 0 0]]
```

**Task 1_3:** Given the 2D numpy array below, extract and print the second row, the third column, and the submatrix consisting of the first two rows and columns.

arr = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])

```
#your answer
#task1_3.py

import numpy as np

arr = np.array([[10, 20, 30],
                [40, 50, 60],
                [70, 80, 90]])

print("Second row:", arr[1])
print("Third column:", arr[:, 2])
print("Submatrix (first 2 rows and cols):")
print(arr[:2, :2])
```

- The expected console output for a successful test run for task1_3.py is as shown below:

```
Second row: [40 50 60]
Third column: [30 60 90]
Submatrix (first 2 rows and cols):
[[10 20]
 [40 50]]
```

**Task 1_4:** Create a 3D numpy array with shape (2, 2, 3) filled with ones. Then assign the value 5 to the element at position `[1, 0, 2]` and print the entire array.

```
#your answer
#task1_4.py

import numpy as np

arr = np.ones((2, 2, 3), dtype=int)
arr[1, 0, 2] = 5
print(arr)
```

- The expected console output for a successful test run for task1_4.py is as shown below:

```
[[[1 1 1]
  [1 1 1]]

 [[1 1 5]
  [1 1 1]]]
```

**Task 1_5:** Create a numpy array of shape (5,) with values [2, 4, 6, 8, 10]. Using slicing, print every other element starting from the first element.

```
#your answer
#task1_5.py

import numpy as np

arr = np.array([2, 4, 6, 8, 10])
print(arr[::2])
```

- The expected console output for a successful test run for task1_5.py is as shown below:

```
[ 2  6 10]
```

**Task 1_6:** Given two numpy arrays below, compute and print their element-wise sum, difference, product, and division.
a = np.array([5, 10, 15])
b = np.array([2, 4, 6])

```
#your answer

import numpy as np

a = np.array([5, 10, 15])
b = np.array([2, 4, 6])
print("Sum:", a + b)
print("Difference:", a - b)
print("Product:", a * b)
print("Division:", a / b)
```

- The expected console output for a successful test run for task1_6.py is as shown below:

```
Sum: [ 7 14 21]
Difference: [3 6 9]
Product: [10 40 90]
Division: [2.5 2.5 2.5]
```

**Task 1_7:** Create a numpy matrix of shape (3, 3) filled with values from 1 to 9.
Create a 1D numpy array of length 3 with values [1, 0, -1].
Multiply the matrix by this array using broadcasting and print the result.

```
#your answer

import numpy as np

matrix = np.arange(1, 10).reshape(3, 3)
vector = np.array([1, 0, -1])

result = matrix * vector
print(result)
```

- The expected console output for a successful test run for task1_7.py is as shown below:

```
[[ 1  0 -3]
 [ 4  0 -6]
 [ 7  0 -9]]
```

**Task 1_8:** Create a numpy array using `arange` method of numpy package, from 0 to 15 with a step of 3. Then compute and print the square root of each element.

```
#your answer

import numpy as np

arr = np.arange(0, 16, 3)
sqrt_arr = np.sqrt(arr)
print(sqrt_arr)
```

- The expected console output for a successful test run for task1_8.py is as shown below:

```
[0.
1.73205081 2.44948974 3.
3.46410162 3.87298335]
```

**Task 1_9:** Given the 3D numpy array of temperatures below (shape: (2, 2, 3)), print the temperatures recorded at the second location (index 1 in second dimension) for all years and all months.

```
temps = np.array([
   [[10, 15, 20], [25, 30, 35]],
   [[11, 16, 21], [26, 31, 36]]
])
```

```
#your answer

import numpy as np

temps = np.array([
   [[10, 15, 20], [25, 30, 35]],
   [[11, 16, 21], [26, 31, 36]]
])
print (temps[:, 1 ,:])
```

The results for a successful test run for task1_9.py is as shown below:

```
[[25 30 35]
 [26 31 36]]
```

**Task 1_10:** You have the following growth factors and rainfall data for three tree types over three regions and months:

```
growth_factors = np.array([0.7, 0.2, 0.5])
rainfall = np.array([[40, 50, 60],
                     [20, 35, 25],
                     [30, 40, 55]])
```

Calculate the potential height increase for each region over the months by broadcasting multiplication and print the result.

```
#your answer

import numpy as np

growth_factors = np.array([0.7, 0.2, 0.5])
rainfall = np.array([[40, 50, 60],
                     [20, 35, 25],
                     [30, 40, 55]])

height_increase = growth_factors[:, np.newaxis] * rainfall
print(height_increase)
```

The results for a successful test run for task1_10.py is as shown below:

```
[[28.  35.  42. ]
 [ 4.   7.   5. ]
 [15.  20.  27.5]]
```

**Task 2:Introduction to OpenCV package (opencv-python · PyPI )**

In this task you will learn about OpenCV package, which is similar to Pillow/PIL library and requires installation before using it to write programs similar to the PIL package

- Go through the documentation for OpenCV (opencv-python · PyPI ) to understand what this package can do.
- OpenCV needs to be installed in your python programming environment as it not included as part of the standard Python library. You need to install it using pip for windows machine using the terminal:

```
>> pip install opencv-python
```

- **Note:**
  After installing opencv, for each of the activity subtask questions, you need to write a python script and test it, or, test the python script (if the script is provided already) for each of the following questions in Task 2, and save the correctly working scripts as : task2_1.py, task2_2.py……..task2_10.py. The scripts (if not included) , the test results and/or screen shots need to be embedded in this document only and submitted to GitHub, after demonstration in the tutorial class to the tutor

**Task 2_1:** Write a Python script to read an image from a file. You would need to use one of the insect images provided in Assets folder in Week 4 tutorial

```
#your answer
#task2_1.py

import cv2

image = cv2.imread('image.png')
print(f"Type of the image: {type(image)}")
print(f"Shape of the image array:
{image.shape}")
```

- The expected console output for a successful test run for task2_1.py is as shown below:

```
Type of the image: <class
'numpy.ndarray'>
Shape of the image array: (2879,
1920, 3)
```

**Task 2_2:** Using OpenCV, split the channels of the image (image.png) and display each color channel in a separate window. Remember OpenCV reads BGR channels.

```
#your answer
#task2_2.py
import cv2

image = cv2.imread('image.png')
b, g, r = cv2.split(image)
cv2.imshow('Blue Channel', b)
cv2.imshow('Green Channel', g)
cv2.imshow('Red Channel', r)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

- The expected console output for a successful test run for task2_2.py is as shown below:

**Task 2_3:** Given a script to resize image.png to 400x300 pixels as shown below, display the resized image.

```
#your answer
#task2_3.py

import cv2

image = cv2.imread('image.png')
resized_img = cv2.resize(image, (400, 300))
cv2.imshow('Resized Image', resized_img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

- The expected console output for a successful test run for task2_3.py is as shown below:

**Task 2_4:** Convert the image image.png to grayscale and save it as gray_image.png
.

```
#your answer
#task2_4.py

import cv2

image = cv2.imread('image.png')
gray_image = cv2.cvtColor(image,
cv2.COLOR_BGR2GRAY)
cv2.imwrite('gray_image.png', gray_image)
```

- The
expected console output for a successful test run for task2_4.py is as shown below:

```
True
```

**Task 2_5:** Draw a red circle of radius 50 pixels centered at (100,100) on a copy of image.png, then display the image.

```
#your answer
#task2_5.py

import cv2

image = cv2.imread('image.png')
image_copy = image.copy()
cv2.circle(image_copy, (100, 100), 50, (0, 0,
255), 3)  # BGR: Red
cv2.imshow('Image with Circle', image_copy)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

- The expected console output for a successful test run for task2_5.py is as shown below:

**Task 2_6:** Access the pixel value at position (50, 50) in image.png , print it, then change this pixel to white [255, 255, 255] and save the modified image as modified.png

```
#your answer
#task2_6.py

import cv2

image = cv2.imread('image.png')
print(f"Original pixel value at (50, 50): {image[50, 50]}")
image[50, 50] = [255, 255, 255]
cv2.imwrite('modified.png', image)
```

- The expected console output for a successful test run for task2_6.py is as shown below:

```
Original pixel value at (50, 50): [216
219 157]
True
```

**Task 2_7:** Invert the colors of the image (image.png)
by subtracting all pixel values from 255, then display both original and inverted images side by side

```
#your answer
#task2_7.py

import cv2
import numpy as np

image = cv2.imread('image.png')
inverted = 255 - image

combined = np.hstack((image, inverted))
cv2.imshow('Original and Inverted', combined)
cv2.waitKey(0)
```

- The expected console output for a successful test run for task2_7.py is as shown below:

**Task 2_8:** Load two different images image.png and image2.png from the assets folder , resize image2.png to the size of image.png. Then blend them using weights 0.6 and 0.4 and display the blended image.

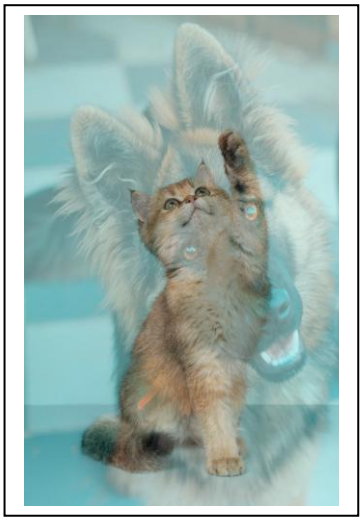
```
#your answer
#task2_8.py

import cv2

img1 = cv2.imread('image.png')
img2 = cv2.imread('image2.png')

img2_resized = cv2.resize(img2, (img1.shape[1], img1.shape[0]))
blended = cv2.addWeighted(img1, 0.6, img2_resized, 0.4, 0)
cv2.imshow('Blended Image', blended)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

- The expected console output for a successful test run for task2_8.py is as shown below:



**Task 2_9:** Perform pixel-wise addition and subtraction of the two resized images from Question 8 using cv2.add() and cv2.subtract(). Display the results in separate windows.

```
#your answer
#task2_9.py
import cv2
img1 = cv2.imread('image.png')
img2 = cv2.imread('image2.png')
img2_resized = cv2.resize(img2, (img1.shape[1], img1.shape[0]))

added = cv2.add(img1, img2_resized)
subtracted = cv2.subtract(img1, img2_resized)

cv2.imshow('Added Image', added)
cv2.imshow('Subtracted Image', subtracted)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

The results for a successful test run for task2_9.py is as shown below:

**Task 2_10:** Convert image.png from BGR to HSV color space, then split and display the Hue, Saturation, and Value channels individually.

```
#your answer
#task2_10.py

import cv2

image = cv2.imread('image.png')
hsv_img = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
h, s, v = cv2.split(hsv_img)

cv2.imshow('Hue Channel', h)
cv2.imshow('Saturation Channel', s)
cv2.imshow('Value Channel', v)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

The results for a successful test run for task2_10.py is as shown below